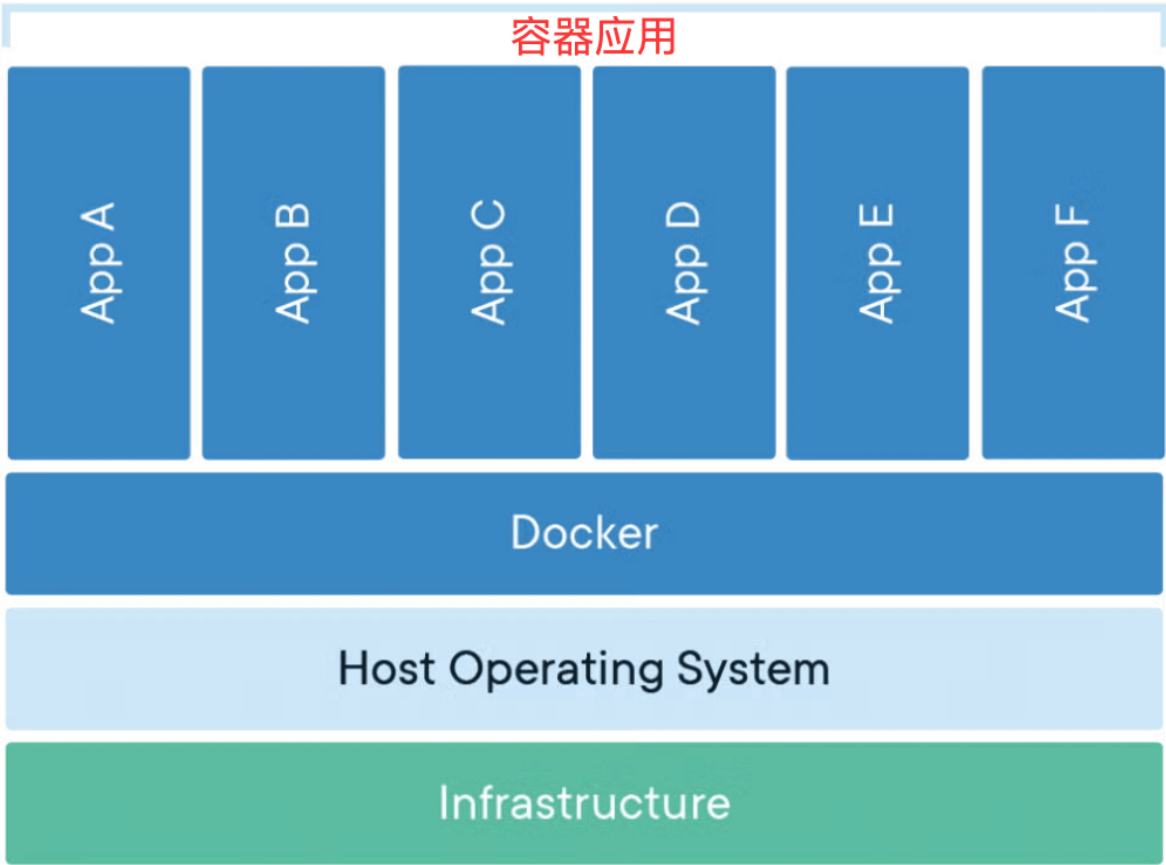


Docker 基础概念

什么是容器化技术

容器共享主机内核，轻量、隔离且高效，不像虚拟机需要完整的操作系统，下图展示了 Docker 容器的基本架构：

- 上层 是多个容器（App A~F），每个容器独立运行一个应用。
- 中间层 是 Docker，负责管理这些容器。
- 底层 是主机操作系统（Host OS）和基础设施，为容器提供硬件和系统支持。



1、传统应用部署的痛点

在传统的应用部署中，我们经常遇到以下问题：

- **环境不一致：**应用在开发环境运行正常，但在测试或生产环境出现问题
- **依赖管理复杂：**不同应用需要不同版本的运行时环境、库文件等
- **资源利用率低：**传统虚拟机需要完整的操作系统，占用大量资源
- **部署复杂：**需要手动配置环境、安装依赖，容易出错

2、容器化技术的解决方案

容器化技术通过以下方式解决了这些问题：

- **环境标准化**：将应用及其依赖打包在一起，确保在任何环境中都能一致运行
- **轻量级**：容器共享宿主机的操作系统内核，比虚拟机更轻量
- **快速部署**：容器可以在几秒内启动，大大提高了部署效率
- **可移植性**：一次构建，到处运行

3、容器化的核心理念

容器化遵循"不可变基础设施"的理念：

- 应用和环境被打包成不可变的镜像
- 每次部署都使用相同的镜像
- 配置通过环境变量或配置文件注入
- 问题修复通过重新构建镜像而非修改运行中的容器

Docker 的核心概念

1、镜像 (Image)

定义：镜像是一个只读的模板，包含了运行应用所需的所有内容：代码、运行时、库文件、环境变量和配置文件。

特点：

- **分层存储**：镜像由多个层组成，每一层代表一次修改
- **只读性**：镜像本身是只读的，不能直接修改
- **可复用**：同一个镜像可以创建多个容器
- **版本管理**：通过标签(tag)进行版本管理

类比理解：镜像就像是一个安装程序或者模板，它定义了应用运行所需的一切，但本身不能直接运行。

2、容器 (Container)

定义：容器是镜像的运行实例，是一个轻量级、可移植的执行环境。

特点：

- **隔离性**：每个容器都有自己的文件系统、网络 and 进程空间
- **临时性**：容器可以被创建、启动、停止、删除
- **可写层**：容器在镜像基础上添加了一个可写层
- **进程级**：容器内通常运行一个主进程

类比理解：如果镜像是类，那么容器就是对象实例。一个镜像可以创建多个容器，就像一个类可以创建多个对象。

3、仓库 (Repository)

定义：仓库是存储和分发镜像的地方，可以包含一个镜像的多个版本。

分类：

- **公共仓库：**如 Docker Hub，任何人都可以使用
- **私有仓库：**企业内部搭建，用于存储私有镜像
- **官方仓库：**由软件官方维护的镜像仓库

Registry vs Repository：

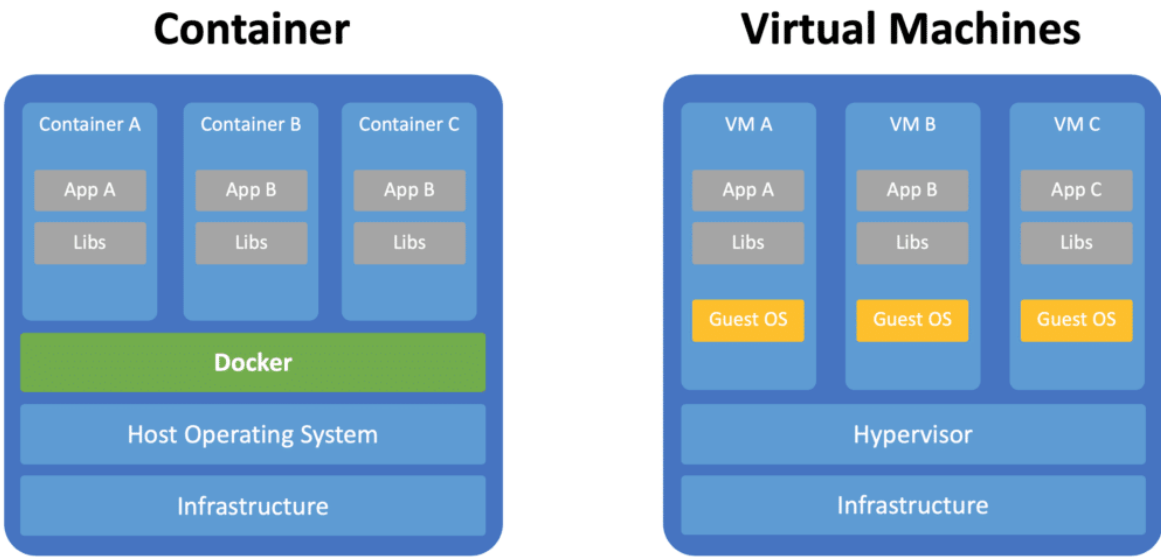
- **Registry：**仓库注册服务器，如 Docker Hub
- **Repository：**具体的镜像仓库，如 nginx、mysql

Docker 与虚拟机的区别

1、架构对比

特性	虚拟机	Docker容器
隔离级别	硬件级别虚拟化	操作系统级别虚拟化
操作系统	每个VM需要完整OS	共享宿主机OS内核
资源占用	重量级，占用较多资源	轻量级，资源占用少
启动时间	分钟级别	秒级别
性能开销	较大	接近原生性能
镜像大小	GB级别	MB级别

2、容器 VS 虚拟机架构



3、使用场景对比

虚拟机适用场景：

- 需要完全隔离的环境
- 运行不同操作系统的应用
- 需要硬件级别的安全隔离

Docker容器适用场景：

- 微服务架构
- CI/CD流水线
- 应用快速部署和扩展
- 开发环境标准化

Docker 的优势和应用场景

主要优势

1. 环境一致性

- **问题解决：**"在我机器上能运行"的问题
- **实现方式：**应用和环境打包在一起
- **价值：**减少环境相关的bug和部署问题

2. 轻量级和高效

- **资源利用：**比虚拟机占用更少资源
- **启动速度：**秒级启动时间
- **密度：**单机可运行更多应用实例

3. 可移植性

- **跨平台：**Linux、Windows、macOS都支持
- **云原生：**在各种云平台间迁移
- **混合环境：**本地开发，云端部署

4. 版本控制和回滚

- **镜像版本：**每个版本都有对应的镜像
- **快速回滚：**出问题时快速回到上一版本
- **A/B测试：**同时运行不同版本进行对比

5. 扩展性

- **水平扩展：**快速创建更多容器实例
- **弹性伸缩：**根据负载自动调整容器数量
- **微服务：**服务拆分和独立部署

典型应用场景

1. 微服务架构

- **服务拆分**：每个微服务独立容器化
- **独立部署**：服务可以独立更新和扩展
- **技术栈自由**：不同服务可以使用不同技术

2. CI/CD流水线

- **构建环境**：标准化的构建环境
- **测试隔离**：每个测试在独立容器中运行
- **部署一致性**：相同镜像在不同环境部署

3. 开发环境标准化

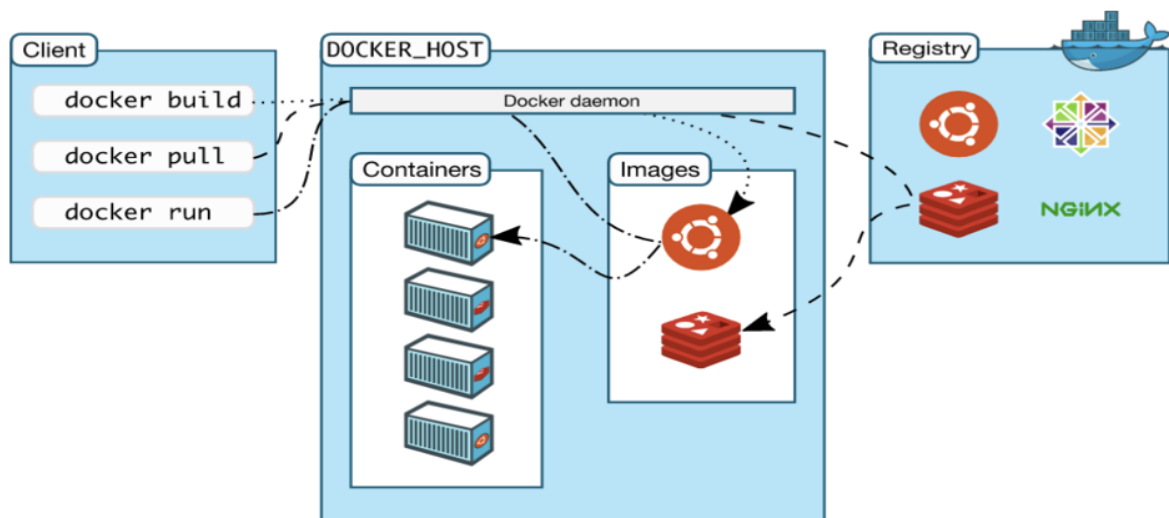
- **快速搭建**：新成员快速获得开发环境
- **版本同步**：团队使用相同的开发环境
- **依赖管理**：避免本地环境冲突

4. 应用现代化

- **遗留系统**：将传统应用容器化
- **云迁移**：帮助应用迁移到云平台
- **混合云**：在不同云环境间移植

Docker 架构组件

整体架构图



Docker Client

功能：

- 用户与Docker交互的主要方式
- 接收用户命令并发送给Docker Daemon

- 可以与远程Docker Daemon通信

常用命令：

- `docker run` - 运行容器
- `docker build` - 构建镜像
- `docker pull` - 拉取镜像
- `docker ps` - 查看容器状态

Docker Daemon

功能：

- Docker的核心服务进程
- 管理镜像、容器、网络 and 存储卷
- 监听Docker API请求并处理

主要职责：

- 镜像管理（构建、存储、分发）
- 容器生命周期管理
- 网络管理
- 数据卷管理
- 与Registry通信

Docker Engine

组成：

- Docker Client + Docker Daemon + REST API
- 是Docker的核心组件

工作流程：

1. Client发送命令到Daemon
2. Daemon解析并执行命令
3. 与Registry交互（如需要）
4. 管理本地镜像和容器
5. 返回结果给Client

Docker Registry

作用：

- 存储和分发Docker镜像
- 提供镜像的版本管理
- 支持公有和私有仓库

Docker Hub特点：

- 官方公共Registry

- 包含大量预构建镜像
- 支持自动构建功能
- 免费和付费服务

Docker 的发展历史

关键时间节点

- **2013年**：Docker开源发布
- **2014年**：Docker 1.0发布，生产环境可用
- **2016年**：Docker Swarm发布，内置编排功能
- **2017年**：Docker分为CE(社区版)和EE(企业版)
- **2019年**：Docker Desktop发布，改善开发者体验

生态系统发展

- **容器编排**：Kubernetes成为事实标准
- **容器运行时**：containerd、CRI-O等替代方案
- **镜像格式**：OCI标准制定
- **安全工具**：容器安全扫描和监控工具

Docker 架构

Docker 架构是基于客户端-服务器模式的，其中包括多个关键组件，确保容器化应用的高效构建、管理和运行。

Docker 的架构设计使得开发者能够轻松地将应用程序与其所有依赖封装在一个可移植的容器中，并在不同的环境中一致地运行。

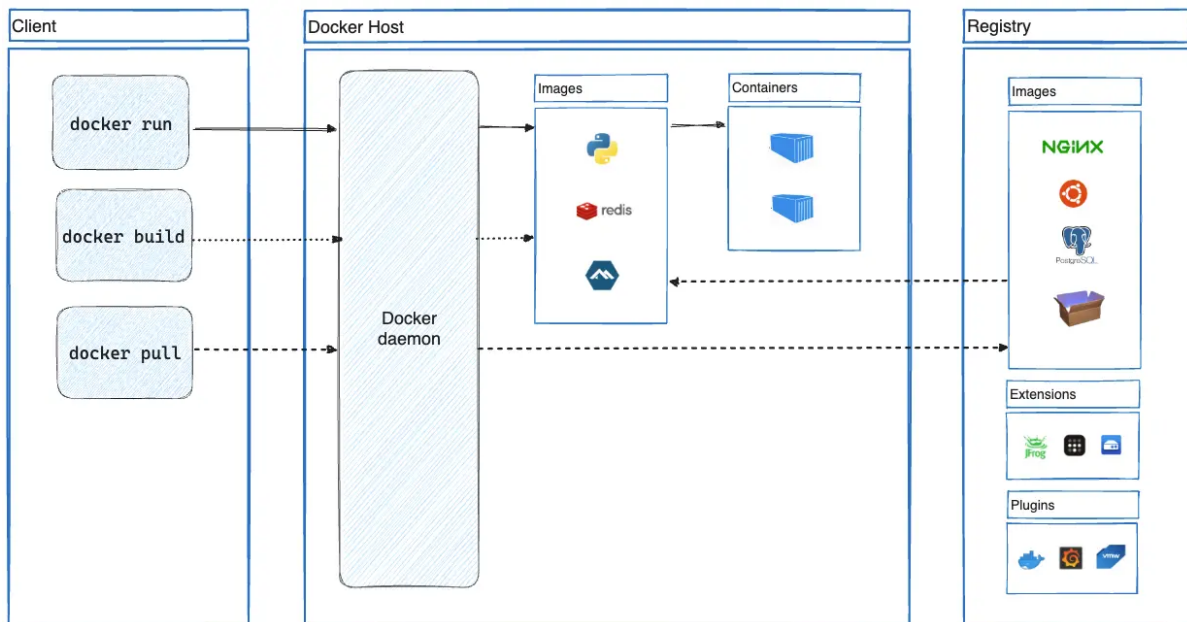
Docker 使用客户端-服务器 (C/S) 架构模式，使用远程 API 来管理和创建 Docker 容器。

Docker 容器通过 Docker 镜像来创建。

容器与镜像的关系类似于面向对象编程中的对象与类。

Docker	面向对象
容器	对象
镜像	类

Docker 架构示意图



Docker 架构的工作流程

- **构建镜像:** 使用 `Dockerfile` 创建镜像。
- **推送镜像到注册表:** 将镜像上传到 Docker Hub 或私有注册表中。
- **拉取镜像:** 通过 `docker pull` 从注册表中拉取镜像。
- **运行容器:** 使用镜像创建并启动容器。
- **管理容器:** 使用 Docker 客户端命令管理正在运行的容器（例如查看日志、停止容器、查看资源使用情况等）。
- **网络与存储:** 容器之间通过 Docker 网络连接，数据通过 Docker 卷或绑定挂载进行持久化。

接下来让我们深入探讨 Docker 的核心组件及其工作机制。

1、Docker 客户端 (Docker Client)

Docker 客户端是用户与 Docker 守护进程交互的命令行界面 (CLI)。它是用户与 Docker 系统的主要交互方式，用户通过 Docker CLI 发出命令，这些命令被发送到 Docker 守护进程，由守护进程执行相应的操作。

- **功能:** 允许用户使用命令与 Docker 守护进程通信，如创建容器、构建镜像、查看容器状态等。
- **交互方式:** Docker 客户端与 Docker 守护进程之间通过 REST API 或 Unix 套接字通信。常用的命令行工具是 `docker`，通过它，用户可以发出各种 Docker 操作命令。

常用命令：

- `docker run`：运行容器。
- `docker ps`：列出正在运行的容器。
- `docker build`：构建 Docker 镜像。
- `docker exec`：在容器中执行命令。

2、Docker 守护进程 (Docker Daemon)

Docker 守护进程 (通常是 `dockerd`) 是 Docker 架构的核心, 负责管理容器生命周期、构建镜像、分发镜像等任务。

守护进程通常以后台进程的方式运行, 等待来自 Docker 客户端的 API 请求。

功能:

- 启动和停止容器。
- 构建、拉取和推送镜像。
- 管理容器的网络和存储。
- 启动、停止、查看容器日志等。
- 与 Docker 注册表进行通信, 管理镜像的存储与分发。

Docker 守护进程监听来自 Docker 客户端的请求, 并且通过 Docker API 执行这些请求。守护进程将负责容器、镜像等 Docker 对象的管理, 并根据请求的参数启动容器、删除容器、修改容器配置等。

启动 Docker 守护进程 (通常是自动启动的) :

```
systemctl start docker
```

3、Docker 引擎 API (Docker Engine API)

Docker 引擎 API 是 Docker 提供的 RESTful 接口, 允许外部客户端与 Docker 守护进程进行通信。通过这个 API, 用户可以执行各种操作, 如启动容器、构建镜像、查看容器状态等。API 提供了 HTTP 请求的接口, 支持跨平台调用。

功能:

- 向 Docker 守护进程发送 HTTP 请求, 实现容器、镜像的管理。
- 提供 RESTful 接口, 允许通过编程与 Docker 进行交互。

可以通过 `curl` 或其他 HTTP 客户端访问 Docker 引擎 API。例如, 查询当前 Docker 守护进程的版本:

```
curl --unix-socket /var/run/docker.sock http://localhost/version
```

4、Docker 容器 (Docker Containers)

容器是 Docker 的执行环境, 它是轻量级、独立且可执行的软件包。容器是从 Docker 镜像启动的, 包含了运行某个应用程序所需的一切——从操作系统库到应用程序代码。容器在运行时与其他容器和宿主机共享操作系统内核, 但容器之间的文件系统和进程是隔离的。

功能:

- 提供独立的运行环境, 确保应用程序在不同的环境中具有一致的行为。
- 容器是临时的, 通常在任务完成后被销毁。

容器的生命周期是由 Docker 守护进程管理的。容器可以在任何地方运行, 因为它们不依赖于底层操作系统的配置, 所有的运行时依赖已经封装在镜像中。

启动一个容器:

```
docker run -d ubuntu
```

5、Docker 镜像 (Docker Images)

Docker 镜像是容器的只读模板。每个镜像都包含了应用程序运行所需的操作系统、运行时、库、环境变量和应用代码等。镜像是静态的，用户可以根据镜像启动容器。

功能：

- 镜像是构建容器的基础，每个容器实例化时都会使用镜像。
- 镜像是只读的，不同容器使用同一个镜像时，容器中的文件系统层是独立的。

Docker 镜像可以通过 `docker pull` 从 Docker Hub 或私有注册表拉取，也可以通过 `docker build` 从 Dockerfile 构建。

拉取 Ubuntu 镜像：

```
docker pull ubuntu
```

6. Docker 仓库 (Docker Registries)

Docker 仓库是用来存储 Docker 镜像的地方，最常用的公共仓库是 **Docker Hub**。用户可以从 Docker Hub 下载镜像，也可以上传自己的镜像分享给其他人。除了公共仓库，用户也可以部署自己的私有 Docker 仓库来管理企业内部的镜像。

功能：

- 存储 Docker 镜像。
- 提供镜像的上传和下载功能。

Docker Hub 提供了大量官方和社区维护的镜像，如 Ubuntu、Nginx、MySQL 等。

推送镜像到 Docker Hub：

```
docker push <username>/<image_name>
```

7、Docker Compose

Docker Compose 是一个用于定义和运行多容器 Docker 应用的工具。通过 Compose，用户可以使用一个 `docker-compose.yml` 配置文件定义多个容器（服务），并可以通过一个命令启动这些容器。

Docker Compose 主要用于开发、测试和部署多容器的应用。

功能：

- 定义和运行多个容器组成的应用。
- 通过 YAML 文件来配置应用的服务、网络 and 卷等。

创建一个简单的 `docker-compose.yml` 文件来配置一个包含 Web 服务和数据库服务的应用：

```
version: '3'
services:
  web:
    image: nginx
    ports:
      - "8080:80"
  db:
    image: mysql
    environment:
      MYSQL_ROOT_PASSWORD: example
```

启动 Compose 定义的所有服务：

```
docker-compose up
```

8、Docker Swarm

Docker Swarm 是 Docker 提供的集群管理和调度工具。它允许将多个 Docker 主机（节点）组织成一个集群，并通过 Swarm 集群管理工具来调度和管理容器。Swarm 可以实现容器的负载均衡、高可用性和自动扩展等功能。

功能：

- 管理多节点 Docker 集群。
- 通过调度器管理容器的部署和扩展。

初始化 Swarm 集群：

```
docker swarm init
```

9、Docker 网络 (Docker Networks)

Docker 网络允许容器之间相互通信，并与外部世界进行连接。Docker 提供了多种网络模式来满足不同需求，如 `bridge` 网络（默认）、`host` 网络和 `overlay` 网络等。

功能：

- 管理容器间的网络通信。
- 支持不同的网络模式，以适应不同场景下的需求。

创建一个自定义网络并将容器连接到该网络：

```
docker network create my_network
docker run -d --network my_network ubuntu
```

10. Docker 卷 (Docker Volumes)

Docker 卷是一种数据持久化机制，允许数据在容器之间共享，并且独立于容器的生命周期。与容器文件系统不同，卷的内容不会随着容器的销毁而丢失，适用于数据库等需要持久存储的应用。

功能：

- 允许容器间共享数据。

- 保证数据持久化，独立于容器的生命周期。

创建并挂载卷：

```
docker volume create my_volume
docker run -d -v my_volume:/data ubuntu
```

CentOS Docker 安装

使用官方安装脚本自动安装

安装命令如下：

```
curl -fsSL https://get.docker.com -o install-docker.sh
sudo sh install-docker.sh
```

手动安装

移除旧版本docker

```
sudo yum remove docker \
docker-client \
docker-client-latest \
docker-common \
docker-latest \
docker-latest-logrotate \
docker-logrotate \
docker-engine
```

配置docker yum源。

```
##安装 yum-utils 工具集，其中包含 yum-config-manager 命令，用于管理 yum 的软件源配置。*
yum install -y yum-utils
##将 Docker 的阿里云镜像源添加到系统的 yum 配置中，后续可以通过 yum install docker-ce 安装 Docker。*
##阿里云提供的 Docker CE 软件源配置文件 URL（包含 CentOS/RHEL 的 Docker 包信息）。*

##为什么使用阿里云镜像源？*
##国内加速：默认的 Docker 官方源可能访问较慢，阿里云镜像源能显著提升下载速度。*
##兼容性：阿里云镜像与官方源保持同步，但更适合国内网络环境。*
yum-config-manager --add-repo http://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo
```

安装 docker

```
sudo yum install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

安装成功后，启动 Docker 引擎：

```
systemctl start docker
systemctl status docker
```

如果希望 Docker 在系统启动时也启动可以使用以下命令：

```
systemctl enable --now docker
```

以上命令会配置 Docker 的 systemd 服务，在系统启动时自动启动 Docker。Docker 安装完默认未启动。并且已经创建好 docker 用户组，但该用户组下没有用户。

运行以下命令来验证安装是否成功：

```
docker run hello-world
docker run library/hello-world:latest
docker pull library/hello-world:latest
```

此命令会下载一个测试镜像，并在容器中运行，当容器运行时，会打印确认消息并退出。

卸载 docker

删除安装包：

```
yum remove docker-ce
```

删除镜像、容器、配置文件等内容：

```
rm -rf /var/lib/docker
```

Docker 镜像使用

当运行容器时，使用的镜像如果在本地中不存在，docker 就会自动从 docker 镜像仓库中下载，默认是从 Docker Hub 公共镜像源下载。

下面我们来学习：

- 1、管理和使用本地 Docker 主机镜像
 - 2、创建镜像
-

列出镜像列表

我们可以使用 **docker images** 来列出本地主机上的镜像。

```
runoob@runoob:~$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED
ubuntu	14.04	90d5884b1ee0	5 days ago
188 MB			
php	5.6	f40e9e0f10c8	9 days ago
444.8 MB			
nginx	latest	6f8d099c3adc	12 days ago
182.7 MB			
mysql	5.6	f2e8d6c772c0	3 weeks ago
324.6 MB			
httpd	latest	02ef73cf1bc0	3 weeks ago
194.4 MB			
ubuntu	15.10	4e3b13c8a266	4 weeks ago
136.3 MB			
hello-world	latest	690ed74de00f	6 months ago
960 B			
training/webapp	latest	6fae60ef3446	11 months ago
348.8 MB			

各个选项说明:

- **REPOSITORY**: 表示镜像的仓库源
- **TAG**: 镜像的标签
- **IMAGE ID**: 镜像ID
- **CREATED**: 镜像创建时间
- **SIZE**: 镜像大小

同一仓库源可以有多个 TAG，代表这个仓库源的不同个版本，如 ubuntu 仓库源里，有 15.10、14.04 等多个不同的版本，我们使用 REPOSITORY:TAG 来定义不同的镜像。

所以，我们如果要使用版本为15.10的ubuntu系统镜像来运行容器时，命令如下：

```
runoob@runoob:~$ docker run -t -i ubuntu:15.10 /bin/bash
root@d77ccb2e5cca:/#
```

参数说明：

- **-i**: 交互式操作。
- **-t**: 终端。
- **ubuntu:15.10**: 这是指用 ubuntu 15.10 版本镜像为基础来启动容器。
- **/bin/bash**: 放在镜像名后的是命令，这里我们希望有个交互式 Shell，因此用的是 /bin/bash。

如果要使用版本为 14.04 的 ubuntu 系统镜像来运行容器时，命令如下：

```
runoob@runoob:~$ docker run -t -i ubuntu:14.04 /bin/bash
root@39e968165990:/#
```

如果不指定一个镜像的版本标签，例如只使用 ubuntu，docker 将默认使用 ubuntu:latest 镜像。

获取一个新的镜像

当我们在本地主机上使用一个不存在的镜像时 Docker 就会自动下载这个镜像。如果我们想预先下载这个镜像，我们可以使用 docker pull 命令来下载它。

```
Crunoob@runoob:~$ docker pull ubuntu:13.10
13.10: Pulling from library/ubuntu
6599cadaf950: Pull complete
23eda618d451: Pull complete
f0be3084efe9: Pull complete
52de432f084b: Pull complete
a3ed95caeb02: Pull complete
Digest: sha256:15b79a6654811c8d992ebacdfbd5152fcf3d165e374e264076aa435214a947a3
Status: Downloaded newer image for ubuntu:13.10
```

下载完成后，我们可以直接使用这个镜像来运行容器。

查找镜像

我们可以从 Docker Hub 网站来搜索镜像，Docker Hub 网址为：<https://hub.docker.com/>

我们也可以使用 docker search 命令来搜索镜像。比如我们需要一个 httpd 的镜像来作为我们的 web 服务。我们可以通过 docker search 命令搜索 httpd 来寻找适合我们的镜像。

也可以使用国内的<https://1ms.run/> 毫秒镜像

```
docker search httpd
```

点击图片查看大图：

NAME	DESCRIPTION	STARS	OFFICIAL	AUTOMATED
httpd	The Apache HTTP Server Project	2703	[OK]	
centos/httpd-24-centos7	Platform for running Apache httpd 2.4 or bui...	26		
centos/httpd		26		[OK]
armhf/httpd	The Apache HTTP Server Project	8		
arm64v8/httpd	The Apache HTTP Server Project	4		
salim1983hoop/httpd24	Dockerfile running apache config	2		[OK]
lead4good/httpd-fpm	httpd server which connects via fcgi proxy h...	1		[OK]
appertly/httpd	Customized Apache HTTPD that uses a PHP-FPM ...	0		[OK]
itsziget/httpd24	Extended HTTPD Docker image based on the off...	0		[OK]
dockerpinata/httpd		0		

NAME: 镜像仓库源的名称

DESCRIPTION: 镜像的描述

OFFICIAL: 是否 docker 官方发布

stars: 类似 Github 里面的 star，表示点赞、喜欢的意思。

AUTOMATED: 自动构建。

拖取镜像

我们决定使用上图中的 httpd 官方版本的镜像，使用命令 docker pull 来下载镜像。

```
docker pull httpd

Using default tag: latest
latest: Pulling from library/httpd
8b87079b7a06: Pulling fs layer
a3ed95caeb02: Download complete
0d62ec9c6a76: Download complete
a329d50397b9: Download complete
ea7c1f032b5c: waiting
be44112b72c7: waiting
```

下载完成后，我们就可以使用这个镜像了。

```
docker run httpd
```

删除镜像

镜像删除使用 **docker rmi** 命令，比如我们删除 hello-world 镜像：

```
docker rmi hello-world
```

```
$ docker rmi hello-world
Untagged: hello-world:latest
Untagged: hello-world@sha256:c3b4ada4687bbaa170745b3e4dd8ac3f194ca95b2d0518b417fb47e5879d9b5f
Deleted: sha256:fce289e99eb9bca977dae136fbe2a82b6b7d4c372474c9235adc1741675f587e
Deleted: sha256:af0b15c8625bb1938f1d7b17081031f649fd14e6b233688eea3c5483994a66a3
```

Docker 容器使用

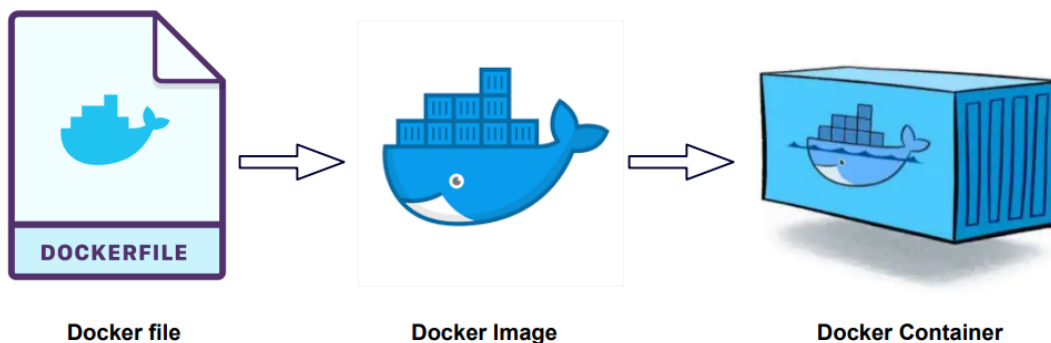
Docker 容器是一个轻量级、可移植、自给自足的软件环境，用于运行应用程序。

Docker 容器将应用程序及其所有依赖项（包括库、配置文件、系统工具等）封装在一个标准化的包中，使得应用能够在任何地方一致地运行。

容器是操作系统级别的虚拟化，不需要运行完整的操作系统，启动和运行更为高效。

镜像与容器的关系

- **镜像 (Image)**：容器的静态模板，包含了应用程序运行所需的所有依赖和文件。镜像是不可变的。
- **容器 (Container)**：镜像的一个运行实例，具有自己的文件系统、进程、网络等，且是动态的。容器从镜像启动，并在运行时保持可变。



Docker 客户端

Docker 客户端是与 Docker 守护进程（Docker Daemon）交互的命令行工具。

docker 客户端非常简单，我们可以直接输入 docker 命令来查看到 Docker 客户端的所有命令选项。

```
docker
```

```
runoob@runoob:/root$ docker
Usage: docker [OPTIONS] COMMAND [arg...]
       docker daemon [ --help | ... ]
       docker [ --help | -v | --version ]

A self-sufficient runtime for containers.

Options:
  --config=~/docker          Location of client config files
  -D, --debug                Enable debug mode
  -H, --host=[]              Daemon socket(s) to connect to
  -h, --help                 Print usage
  -l, --log-level=info       Set the logging level
  --tls                      Use TLS; implied by --tlsverify
  --tlscacert=~/docker/ca.pem Trust certs signed only by this CA
  --tlscert=~/docker/cert.pem Path to TLS certificate file
  --tlskey=~/docker/key.pem  Path to TLS key file
  --tlsverify                Use TLS and verify the remote
  -v, --version              Print version information and quit

Commands:
  attach    Attach to a running container
  build     Build an image from a Dockerfile
  commit    Create a new image from a container's changes
  cp        Copy files/folders between a container and the local filesystem
  create    Create a new container
  diff      Inspect changes on a container's filesystem
```

可以通过命令 **docker command --help** 更深入的了解指定的 Docker 命令使用方法。

例如我们要查看 **docker stats** 指令的具体使用方法：

```
docker stats --help
```

```
runoob@runoob:/root$ docker stats --help
Usage:  docker stats [OPTIONS] [CONTAINER...]

Display a live stream of container(s) resource usage statistics

  -a, --all          Show all containers (default shows just running)
  --help             Print usage
  --no-stream        Disable streaming stats and only pull the first result
```

以下是常用的 Docker 客户端命令：

命令	功能	示例
<code>docker run</code>	启动一个新的容器并运行命令	<code>docker run -d ubuntu</code>
<code>docker ps</code>	列出当前正在运行的容器	<code>docker ps</code>
<code>docker ps -a</code>	列出所有容器（包括已停止的容器）	<code>docker ps -a</code>
<code>docker build</code>	使用 Dockerfile 构建镜像	<code>docker build -t my-image .</code>
<code>docker images</code>	列出本地存储的所有镜像	<code>docker images</code>
<code>docker pull</code>	从 Docker 仓库拉取镜像	<code>docker pull ubuntu</code>
<code>docker push</code>	将镜像推送到 Docker 仓库	<code>docker push my-image</code>
<code>docker exec</code>	在运行的容器中执行命令	<code>docker exec -it container_name bash</code>
<code>docker stop</code>	停止一个或多个容器	<code>docker stop container_name</code>
<code>docker start</code>	启动已停止的容器	<code>docker start container_name</code>
<code>docker restart</code>	重启一个容器	<code>docker restart container_name</code>
<code>docker rm</code>	删除一个或多个容器	<code>docker rm container_name</code>
<code>docker rmi</code>	删除一个或多个镜像	<code>docker rmi my-image</code>
<code>docker logs</code>	查看容器的日志	<code>docker logs container_name</code>
<code>docker inspect</code>	获取容器或镜像的详细信息	<code>docker inspect container_name</code>
<code>docker exec -it</code>	进入容器的交互式终端	<code>docker exec -it container_name /bin/bash</code>
<code>docker network ls</code>	列出所有 Docker 网络	<code>docker network ls</code>
<code>docker volume ls</code>	列出所有 Docker 卷	<code>docker volume ls</code>
<code>docker-compose up</code>	启动多容器应用（从 <code>docker-compose.yml</code> 文件）	<code>docker-compose up</code>
<code>docker-compose down</code>	停止并删除由 <code>docker-compose</code> 启动的容器、网络等	<code>docker-compose down</code>
<code>docker info</code>	显示 Docker 系统的详细信息	<code>docker info</code>
<code>docker version</code>	显示 Docker 客户端和守护进程的版本信息	<code>docker version</code>
<code>docker stats</code>	显示容器的实时资源使用情况	<code>docker stats</code>

命令	功能	示例
<code>docker login</code>	登录 Docker 仓库	<code>docker login</code>
<code>docker logout</code>	登出 Docker 仓库	<code>docker logout</code>

常用选项说明:

- `-d`: 后台运行容器, 例如 `docker run -d ubuntu`。
- `-it`: 以交互式终端运行容器, 例如 `docker exec -it container_name bash`。
- `-t`: 为镜像指定标签, 例如 `docker build -t my-image .`。

容器使用

获取镜像

如果我们本地没有 ubuntu 镜像, 我们可以使用 `docker pull` 命令来载入 ubuntu 镜像:

```
docker pull ubuntu
```

启动容器

以下命令使用 ubuntu 镜像启动一个容器, 参数为以命令行模式进入该容器:

```
docker run -it ubuntu /bin/bash
```

```
$ docker run -it ubuntu /bin/bash
root@b750bbbcfd88:/#
```

参数说明:

- `-i`: 交互式操作。
- `-t`: 终端。
- **ubuntu**: ubuntu 镜像。
- **/bin/bash**: 放在镜像名后的是命令, 这里我们希望有个交互式 Shell, 因此用的是 `/bin/bash`。

要退出终端, 直接输入 **exit**:

```
root@ed09e4490c57:/# exit
```

```
$ docker run -it ubuntu /bin/bash
root@b750bbbcfd88:/# exit
exit
```

启动已停止运行的容器

查看所有的容器命令如下:

```
$ docker ps -a
```

点击图片查看大图:

```
$ docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
b750bbbcfd88        ubuntu             "/bin/bash"        10 minutes ago     Exited (0) 2 minutes ago
```

使用 `docker start` 启动一个已停止的容器：

```
$ docker start b750bbbcfd88
```

```
$ docker start b750bbbcfd88
b750bbbcfd88
```

后台运行

在大部分的场景下，我们希望 `docker` 的服务是在后台运行的，我们可以过 `-d` 指定容器的运行模式。

```
$ docker run -itd --name ubuntu-test ubuntu /bin/bash
```

点击图片查看大图：

```
$ docker run -itd --name ubuntu-test ubuntu /bin/bash
1e560fca39064157ba003abbdfe2a4dafc77e68a824c38ff0a2104327ca08cb6
```

```
$ docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
1e560fca3906        ubuntu             "/bin/bash"        9 seconds ago      Up 9 seconds
```

注：加了 `-d` 参数默认不会进入容器，想要进入容器需要使用指令 `docker exec`（下面会介绍到）。

停止一个容器

停止容器的命令如下：

```
$ docker stop <容器 ID>
```

```
$ docker stop 1e560fca3906
1e560fca3906
```

停止的容器可以通过 `docker restart` 重启：

```
$ docker restart <容器 ID>
```

```
$ docker restart 1e560fca3906
1e560fca3906
```

进入容器

在使用 `-d` 参数时启动容器时，容器会运行在后台，这时如果要进入容器，可以通过以下命令进入：

- **docker attach**：允许你与容器的标准输入（stdin）、输出（stdout）和标准错误（stderr）进行交互。
 - **注意：** 如果从这个容器退出，会导致容器的停止。
- **docker exec**：推荐大家使用 `docker exec` 命令，因为此命令会退出容器终端，但不会导致容器的停止。

exec 命令

使用 `docker exec` 命令实例：

```
docker exec -it 243c32535da7 /bin/bash
```

```
$ docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
1e560fca3906        ubuntu             "/bin/bash"        19 minutes ago     Up 3 seconds                       ubuntu-test

LINSHAODONG@HTJT-LSDONG MINGW64 /e/project/swoft-master
$ docker exec -it 1e560fca3906 /bin/bash
root@1e560fca3906:/# exit
exit

LINSHAODONG@HTJT-LSDONG MINGW64 /e/project/swoft-master
$ docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
1e560fca3906        ubuntu             "/bin/bash"        20 minutes ago     Up 34 seconds                       ubuntu-test
```

注意：如果从这个容器退出，容器不会停止，这就是为什么推荐大家使用 **docker exec**。

更多参数说明请使用 **docker exec --help** 命令查看。

删除容器

删除容器使用 **docker rm** 命令：

```
$ docker rm -f 1e560fca3906
```

```
$ docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
1e560fca3906        ubuntu             "/bin/bash"        29 minutes ago     Up 9 minutes                       ubuntu-test

LINSHAODONG@HTJT-LSDONG MINGW64 /e/project
$ docker rm -f 1e560fca3906
1e560fca3906

LINSHAODONG@HTJT-LSDONG MINGW64 /e/project
$ docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
1e560fca3906        ubuntu             "/bin/bash"        29 minutes ago     Up 9 minutes                       ubuntu-test

LINSHAODONG@HTJT-LSDONG MINGW64 /e/project
```

下面的命令可以清理掉所有处于终止状态的容器。

```
$ docker container prune
```

运行一个 web 应用

前面我们运行的容器并没有一些什么特别的用处。

接下来让我们尝试使用 docker 构建一个 web 应用程序。

我们将在docker容器中运行一个 Python Flask 应用来运行一个web应用。

```
runoob@runoob:~# docker pull training/webapp # 载入镜像
runoob@runoob:~# docker run -d -P training/webapp python app.py
```

```
runoob@runoob:~$ docker run -d -P training/webapp python app.py
4ecb9ce5113ded09117a94462fee760c89f9db7bfc10365ca20f2d5a4430871b
runoob@runoob:~$
```

参数说明：

- **-d**:让容器在后台运行。
- **-P**:将容器内部使用的网络端口随机映射到我们使用的主机上。

查看 WEB 应用容器

使用 `docker ps` 来查看我们正在运行的容器：

```
runoob@runoob:~# docker ps
```

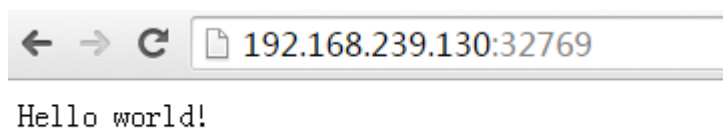
CONTAINER ID	IMAGE	COMMAND	...	PORTS
d3d5e39ed9d3	training/webapp	"python app.py"	...	0.0.0.0:32769->5000/tcp

这里多了端口信息。

```
PORTS
0.0.0.0:32769->5000/tcp
```

Docker 开放了 5000 端口（默认 Python Flask 端口）映射到主机端口 32769 上。

这时我们可以通过浏览器访问WEB应用



我们也可以通过 `-p` 参数来设置不一样的端口：

```
docker run -d -p 5000:5000 training/webapp python app.py

docker run -d -p 80:80 nginx
docker run -d -P nginx
```

docker ps查看正在运行的容器

```
runoob@runoob:~# docker ps
```

CONTAINER ID	IMAGE	PORTS
bf08b7f2cd89	training/webapp	0.0.0.0:5000->5000/tcp
wizardly_chandrasekhar		
d3d5e39ed9d3	training/webapp	0.0.0.0:32769->5000/tcp
xenodochial_hoov		

容器内部的 5000 端口映射到我们本地主机的 5000 端口上。

网络端口的快捷方式

通过 `docker ps` 命令可以查看到容器的端口映射，`docker` 还提供了另一个快捷方式 `docker port`，使用 `docker port` 可以查看指定（ID 或者名字）容器的某个确定端口映射到宿主机的端口号。

查看 WEB 应用程序日志

`docker logs [ID或者名字]` 可以查看容器内部的标准输出。

```
runoob@runoob:~$ docker logs -f bf08b7f2cd89
* Running on http://0.0.0.0:5000/ (Press CTRL+C to quit)
192.168.239.1 - - [09/May/2016 16:30:37] "GET / HTTP/1.1" 200 -
192.168.239.1 - - [09/May/2016 16:30:37] "GET /favicon.ico HTTP/1.1" 404 -
```

-f: 让 **docker logs** 像使用 **tail -f** 一样来输出容器内部的标准输出。

从上面，我们可以看到应用程序使用的是 5000 端口并且能够查看到应用程序的访问日志。

查看WEB应用程序容器的进程

我们还可以使用 **docker top** 来查看容器内部运行的进程

```
runoob@runoob:~$ docker top wizardly_chandrasekhar
UID          PID         PPID        ...        TIME        CMD
root         23245       23228       ...        00:00:00    python app.py
```

检查 WEB 应用程序

使用 **docker inspect** 来查看 Docker 的底层信息。它会返回一个 JSON 文件记录着 Docker 容器的配置和状态信息。

```
runoob@runoob:~$ docker inspect wizardly_chandrasekhar
[
  {
    "Id": "bf08b7f2cd897b5964943134aa6d373e355c286db9b9885b1f60b6e8f82b2b85",
    "Created": "2018-09-17T01:41:26.174228707Z",
    "Path": "python",
    "Args": [
      "app.py"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 23245,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2018-09-17T01:41:26.494185806Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    },
    .....
  ]
}
```

停止 WEB 应用容器

```
docker stop wizardly_chandrasekhar
```

重启WEB应用容器

已经停止的容器，我们可以使用命令 `docker start` 来启动。

```
docker start wizardly_chandrasekhar
```

`docker ps -l` 查询最后一次创建的容器：

```
# docker ps -l
CONTAINER ID        IMAGE               PORTS
NAMES
bf08b7f2cd89       training/webapp    ...    0.0.0.0:5000->5000/tcp
wizardly_chandrasekhar
```

正在运行的容器，我们可以使用 **`docker restart`** 命令来重启。

移除WEB应用容器

我们可以使用 `docker rm` 命令来删除不需要的容器

```
docker rm wizardly_chandrasekhar
wizardly_chandrasekhar
```

删除容器时，容器必须是停止状态，否则会报如下错误

```
runoob@runoob:~$ docker rm wizardly_chandrasekhar
Error response from daemon: You cannot remove a running container
bf08b7f2cd897b5964943134aa6d373e355c286db9b9885b1f60b6e8f82b2b85. Stop the
container before attempting removal or force remove
```

可以添加参数 `-f` 强制移除

```
docker rm -f wizardly_chandrasekhar
```

Docker 目录挂载

一、什么是 Docker 的 Bind Mount？

Bind Mount 是 Docker 提供了一种目录挂载机制，它允许将宿主机上的目录或文件**直接挂载到容器内部**的指定路径。与 Volume 不同，Bind Mount 的路径由用户显式指定，且完全由用户管理（如创建、权限控制等）。

核心特点：

- **路径可控**：用户可以自由选择宿主机上的任意目录进行挂载。
- **实时同步**：宿主机和容器内的文件变化会立即同步（除非挂载为只读）。

- **依赖宿主主机结构**：Bind Mount 的路径必须存在于宿主主机上，否则可能报错（取决于 Docker 版本和配置）。

二、没有 Bind Mount 时，数据会怎样？

场景：运行一个 Nginx 容器，不挂载任何目录。

```
docker run -d --name nginx_no_mount nginx
```

问题：

1. 数据临时性

- Nginx 的默认网页文件（如 `/usr/share/nginx/html/index.html`）存储在容器的可写层中。
- 如果容器被删除，这些文件会永久丢失。

2. 修改困难

- 要修改网页内容，必须进入容器内部编辑文件（如 `docker exec -it nginx_no_mount bash`），操作繁琐且不直观。

验证：

- 访问 `http://localhost`（默认 Nginx 页面）。
- 删除容器后重新运行，发现修改丢失：

```
docker rm -f nginx_no_mount
docker run -d --name nginx_no_mount_2 nginx
```

访问

```
http://localhost
```

，页面恢复默认。

三、有了 Bind Mount 后，数据会怎样？

场景：将宿主机的目录挂载到 Nginx 容器的网页目录，实现数据持久化和实时修改。

步骤 1：准备宿主机目录和文件

```
mkdir -p ~/nginx_html
echo "<h1>Hello from Host via Bind Mount!</h1>" > ~/nginx_html/index.html
```

步骤 2：运行 Nginx 容器并挂载 Bind Mount

```
bashdocker run -d --name nginx_with_bind \
-v ~/nginx_html:/usr/share/nginx/html \
nginx
```

- `-v ~/nginx_html:/usr/share/nginx/html`：将宿主机的 `~/nginx_html` 目录挂载到容器的 `/usr/share/nginx/html`。

验证：

1. 访问网页

- 打开浏览器访问 `http://localhost`，看到 `Hello from Host via Bind Mount!`。

2. 实时修改测试

- 修改宿主机文件：

```
echo "<h1>Updated from Host!</h1>" > ~/nginx_html/index.html
```

- 刷新浏览器，内容立即更新。

3. 容器删除后数据保留

- 删除容器并重新运行：

```
docker rm -f nginx_with_bind
docker run -d --name nginx_with_bind_2 \
-v ~/nginx_html:/usr/share/nginx/html \
nginx
```

- 访问 `http://localhost`，数据仍然存在。

四、Bind Mount 的高级用法

1. 只读挂载

- 防止容器修改宿主机文件：

```
bashdocker run -d --name nginx_ro \
-v ~/nginx_html:/usr/share/nginx/html:ro \
nginx
```

- 容器内尝试修改文件会报错：`Read-only file system`。

2. 挂载单个文件

- 例如修改 Nginx 配置：

```
bashecho "server { listen 8080; }" > ~/nginx.conf
docker run -d --name nginx_custom_conf \
-v ~/nginx.conf:/etc/nginx/conf.d/custom.conf \
nginx
```

五、Bind Mount 与 Volume 的对比

特性	Bind Mount	Volume
管理方	用户	Docker
路径	宿主机任意路径	Docker 自动管理（ <code>/var/lib/docker/volumes/</code> ）
灵活性	高（可挂载任意目录）	低（需通过 Docker 命令管理）
适用场景	开发环境（代码实时同步）	生产环境（数据持久化）
性能	依赖宿主机文件系统	Docker 优化，性能更高

六、总结

1. 没有 Bind Mount

- 数据临时存储在容器内部，删除容器后丢失。
- 修改数据需进入容器，操作繁琐。

2. 有了 Bind Mount

- 数据持久化存储在宿主机上，容器重启或删除后仍保留。
- 支持实时同步，方便开发调试。
- 可通过只读挂载保护宿主机文件。

3. 推荐实践

- **开发环境**：使用 Bind Mount 挂载代码目录，实现实时修改。
- **生产环境**：优先使用 Volume，避免路径依赖和权限问题。

示例完整命令：

```
bash# 准备宿主机目录
mkdir -p ~/nginx_html
echo "<h1>Hello from Bind Mount!</h1>" > ~/nginx_html/index.html

# 运行 Nginx 并挂载 Bind Mount
docker run -d --name nginx_demo \
  -p 80:80 \
  -v ~/nginx_html:/usr/share/nginx/html \
  nginx

# 验证
curl http://localhost
```

注意：

如果宿主机上的目录是空的，容器内对应的挂载目录会被“清空”（即变为空目录）。这是因为 Bind Mount 会完全覆盖容器内的目标目录，以宿主机目录的内容为准。

详细解释

1. Bind Mount 的工作原理

- **Bind Mount** 是将宿主机上的**某个目录或文件**直接挂载到容器内部的指定路径。
- 容器内的目标路径会被**宿主机路径的内容完全覆盖**，而不是合并或递归复制。
- **如果宿主机目录是空的**，容器内的目标目录也会变成空目录（即使容器原本有文件）。

2. 实验验证

实验 1：容器原本有文件，挂载空目录

步骤：

1. 运行一个 Nginx 容器（默认有文件）：

```
/usr/share/nginx/html
```

```
docker run -d --name nginx_original nginx
```

2. 检查容器内的默认网页文件

```
docker exec -it nginx_original ls /usr/share/nginx/html
```

输出

```
50x.html index.html
```

- 说明容器默认有 `index.html` 和 `50x.html`。

3. 在宿主机创建空目录

```
mkdir ~/empty_nginx_html
```

4. 重新运行容器并挂载空目录

```
docker run -d --name nginx_with_empty_mount \  
-v ~/empty_nginx_html:/usr/share/nginx/html \  
nginx
```

5. 检查容器内的目录

```
docker exec -it nginx_with_empty_mount ls /usr/share/nginx/html
```

输出

（无输出，说明目录是空的）

- **结论：**容器内的 `/usr/share/nginx/html` 被宿主机空目录覆盖，文件消失。

实验 2：容器原本有文件，挂载非空目录

步骤：

1. 在宿主机创建非空目录

```
mkdir ~/custom_nginx_html  
echo "<h1>Hello from Host!</h1>" > ~/custom_nginx_html/index.html
```

2. 运行容器并挂载非空目录

```
docker run -d --name nginx_with_custom_mount \  
-v ~/custom_nginx_html:/usr/share/nginx/html \  
nginx
```

3. 检查容器内的目录

```
docker exec -it nginx_with_custom_mount ls /usr/share/nginx/html
```

输出

```
index.html
```

- **结论：**容器内的 `/usr/share/nginx/html` 只包含宿主机目录的文件（`index.html`），原容器内的 `50x.html` 被覆盖。

3. 为什么 Bind Mount 会覆盖容器目录？

- Docker 的设计逻辑
 - **Bind Mount** 是直接将宿主机目录挂载到容器内部，类似于 Linux 的 `mount --bind` 命令。
 - **容器内的目标路径会被宿主机路径完全替换**，而不是合并或递归复制。
 - **如果宿主机目录是空的**，容器内的目录也会变成空目录（即使容器原本有文件）。
- 与 Volume 的区别
 - **Volume** 是 Docker 管理的存储空间，不会直接覆盖容器内的文件，而是合并（如果容器内有同名文件，Volume 会隐藏它）。
 - **Bind Mount** 是直接替换，更“粗暴”。

4. 如何避免 Bind Mount 清空容器目录？

方法 1：先复制容器内的文件到宿主机

1. 运行一个临时容器复制文件到宿主机：

```
docker run --rm --name temp_nginx nginx cat /usr/share/nginx/html/index.html  
> ~/backup_index.html
```

2. 在宿主机创建目录并放入备份文件

```
mkdir ~/safe_nginx_html  
cp ~/backup_index.html ~/safe_nginx_html/
```

3. 挂载 Bind Mount

```
docker run -d --name nginx_safe_mount \  
-v ~/safe_nginx_html:/usr/share/nginx/html \  
nginx
```

- 这样容器内的文件不会被清空，因为宿主机目录有备份。

方法 2：使用 Volume（推荐）

- Volume
 - 不会直接覆盖容器内的文件，而是合并：

```
docker run -d --name nginx_with_volume \
-v nginx_volume:/usr/share/nginx/html \
nginx
```

- 如果 `nginx_volume` 是空的，容器会保留默认文件。
- 如果 `nginx_volume` 有文件，会覆盖容器内的默认文件。

5. 总结

场景	Bind Mount 挂载空目录	Bind Mount 挂载非空目录	Volume
容器内原有文件	✗ 被清空	✗ 被覆盖（仅保留宿主文件）	✓ 保留（除非 Volume 有同名文件）
宿主机目录为空	✓ 容器目录变空	✗ 容器目录变空（如果挂载空目录）	✓ 容器保留默认文件
适用场景	开发环境（需要完全控制容器目录）	开发环境（需要覆盖容器文件）	生产环境（数据持久化）

最佳实践：

- 开发环境：**使用 Bind Mount 挂载代码目录（方便实时修改）。
- 生产环境：**优先使用 Volume（避免意外覆盖文件）。
- 如果必须用 Bind Mount + 空目录，**先备份容器内的文件！

Docker 卷映射

一、什么是 Docker Volume？

Docker Volume 是 Docker 提供的持久化存储机制，用于将容器内的数据存储到宿主机上，实现数据的持久化和共享。与 Bind Mount 不同，Volume 由 Docker 管理，独立于容器的生命周期，即使容器被删除，Volume 中的数据仍然保留。

核心特点：

- 持久化存储：**数据不会因容器删除而丢失。
- 数据共享：**多个容器可以挂载同一个 Volume，实现数据共享。
- 性能优化：**Volume 的性能优于 Bind Mount，尤其在 Linux 主机上。
- 跨平台兼容性：**Volume 的存储路径由 Docker 管理，不依赖宿主机的目录结构。

二、没有 Volume 时，数据会怎样？

场景：运行一个 Nginx 容器，不挂载任何 Volume。

```
docker run -d --name nginx_no_volume nginx
```

问题：

1. 数据临时性

- Nginx 的默认网页文件（如 `/usr/share/nginx/html/index.html`）存储在容器的可写层中。
- 如果容器被删除，这些文件会永久丢失。

2. 修改困难

- 要修改网页内容，必须进入容器内部编辑文件（如 `docker exec -it nginx_no_volume bash`），操作繁琐且不直观。

验证：

- 访问 `http://localhost`（默认 Nginx 页面）。
- 删除容器后重新运行，发现修改丢失：

```
bashdocker rm -f nginx_no_volume
docker run -d --name nginx_no_volume_2 nginx
```

访问

```
http://localhost
```

，页面恢复默认。

三、有了 Volume 后，数据会怎样？

场景：将宿主机的目录或 Docker 管理的 Volume 挂载到 Nginx 容器的网页目录，实现数据持久化和实时修改。

方法 1：使用命名 Volume（推荐）

1. 创建 Volume：

```
docker volume create nginx_volume
```

2. 运行容器并挂载 Volume：

```
docker run -d --name nginx_with_volume \
-v nginx_volume:/usr/share/nginx/html \
nginx
```

3. 验证：

- **访问网页：**打开浏览器访问 `http://localhost`，看到默认 Nginx 页面。
- **修改数据**

```
bashdocker exec -it nginx_with_volume bash
echo "<h1>Hello from volume!</h1>" > /usr/share/nginx/html/index.html
```

刷新浏览器，内容更新。

- 删除容器并重新运行

```
docker rm -f nginx_with_volume
docker run -d --name nginx_with_volume_2 \
  -v nginx_volume:/usr/share/nginx/html \
  nginx
```

访问

<http://localhost>

，数据仍然存在。

方法 2：使用 Bind Mount（开发环境推荐）

1. 准备宿主机目录和文件：

```
bashmkdir -p ~/nginx_html
echo "<h1>Hello from Host via Bind Mount!</h1>" > ~/nginx_html/index.html
```

2. 运行容器并挂载 Bind Mount：

```
bashdocker run -d --name nginx_with_bind \
  -v ~/nginx_html:/usr/share/nginx/html \
  nginx
```

3. 验证：

- **访问网页：**打开浏览器访问 <http://localhost>，看到 `Hello from Host via Bind Mount!`。
- 实时修改测试

```
echo "<h1>Updated from Host!</h1>" > ~/nginx_html/index.html
```

刷新浏览器，内容立即更新。

- 容器删除后数据保留

```
docker rm -f nginx_with_bind
docker run -d --name nginx_with_bind_2 \
  -v ~/nginx_html:/usr/share/nginx/html \
  nginx
```

访问

<http://localhost>

，数据仍然存在。

四、Volume 与 Bind Mount 的对比

特性	Volume	Bind Mount
管理方	Docker	用户
路径	Docker 自动管理 (<code>/var/lib/docker/volumes/</code>)	宿主机任意路径
灵活性	高 (支持命名 Volume 和匿名 Volume)	高 (可挂载任意目录)
适用场景	生产环境 (数据持久化、共享)	开发环境 (代码实时同步)
性能	更高 (优化存储驱动)	依赖宿主机文件系统

五、总结

- 1. 没有 Volume
 - 数据临时存储在容器内部，删除容器后丢失。
 - 修改数据需进入容器，操作繁琐。
- 2. 有了 Volume
 - **数据持久化**：容器删除后，Volume 中的数据仍然保留。
 - **实时同步**：支持双向同步，方便开发调试。
 - **数据共享**：多个容器可以挂载同一个 Volume。
- 3. 推荐实践
 - **生产环境**：使用命名 Volume (如 `-v nginx_volume:/usr/share/nginx/html`) 。
 - **开发环境**：使用 Bind Mount (如 `-v ~/nginx_html:/usr/share/nginx/html`) 实现代码实时同步。

示例完整命令：

```
bash# 创建命名 volume
docker volume create nginx_volume

# 运行 Nginx 并挂载 volume
docker run -d --name nginx_demo \
  -p 80:80 \
  -v nginx_volume:/usr/share/nginx/html \
  nginx

# 验证
curl http://localhost
```

如果创建新的空目录并使用 Docker Volume 挂载，容器内的目录会被清空吗？

与 Bind Mount 不同，Docker Volume 不会直接覆盖容器内的目录内容，而是合并或保留原有文件（具体行为取决于 Volume 是否为空以及容器内是否有同名文件）。

1. 为什么 Volume 不会清空容器目录？

Volume 的工作原理

- **Volume 是 Docker 管理的独立存储空间**，默认存储在 `/var/lib/docker/volumes/` (Linux) 或 Docker 的存储驱动路径下。
- 当挂载 Volume 到容器目录时
 - **如果 Volume 是空的**：容器内的文件会**保留**（但可能被 Volume 的“空状态”隐藏，具体取决于文件系统行为）。
 - **如果 Volume 非空**：Volume 中的文件会**覆盖容器内的同名文件**，但其他文件保留。
- 与 Bind Mount 的区别
 - **Bind Mount** 是直接替换容器目录（类似 `mount --bind`），宿主机目录为空则容器目录变空。
 - **Volume** 是 Docker 管理的存储，行为更“温和”，不会直接清空容器目录。

2. 实验验证

实验 1：挂载空 Volume 到容器目录（容器原本有文件）

步骤：

1. 运行一个 Nginx 容器（默认 `/usr/share/nginx/html` 有文件）

```
docker run -d --name nginx_original nginx
```

2. 检查容器内的默认文件

```
docker exec -it nginx_original ls /usr/share/nginx/html
```

输出

```
50x.html  index.html
```

- 说明容器默认有 `index.html` 和 `50x.html`。

3. 创建一个空 Volume

```
docker volume create empty_volume
```

4. 重新运行容器并挂载空 Volume

```
docker run -d --name nginx_with_empty_volume \  
-v empty_volume:/usr/share/nginx/html \  
nginx
```

5. 检查容器内的目录

```
docker exec -it nginx_with_empty_volume ls /usr/share/nginx/html
```

输出

```
50x.html  index.html
```

- **结论：**容器内的文件**未被清空**，仍然保留默认文件。

实验 2：挂载非空 Volume 到容器目录（覆盖同名文件）

步骤：

1. 创建一个非空 Volume（并添加自定义文件）

```
docker volume create custom_volume
docker run --rm -v custom_volume:/data alpine sh -c "echo 'Hello from
volume!' > /data/index.html"
```

- 这里用临时容器在 `custom_volume` 中创建了 `index.html`。

2. 运行 Nginx 并挂载非空 Volume

```
docker run -d --name nginx_with_custom_volume \
-v custom_volume:/usr/share/nginx/html \
nginx
```

3. 检查容器内的目录

```
docker exec -it nginx_with_custom_volume ls /usr/share/nginx/html
```

输出

```
index.html
```

- 进一步检查文件内容

```
docker exec -it nginx_with_custom_volume cat
/usr/share/nginx/html/index.html
```

输出

```
Hello from volume!
```

- 结论
 - Volume 中的 `index.html` **覆盖了容器默认的** `index.html`。
 - 容器默认的 `50x.html` **被隐藏**（因为 Volume 中没有它）。

3. 为什么 Volume 和 Bind Mount 行为不同？

特性	Volume	Bind Mount
存储位置	Docker 管理 (<code>/var/lib/docker/volumes/</code>)	宿主机任意路径（如 <code>~/my_dir</code> ）

特性	Volume	Bind Mount
空目录挂载行为	保留容器内文件（除非 Volume 有同名文件）	直接替换为空目录（容器内文件消失）
文件覆盖规则	仅覆盖同名文件	完全替换容器目录
适用场景	生产环境（数据持久化）	开发环境（代码实时同步）

4. 总结

场景	Volume 挂载空目录	Bind Mount 挂载空目录
容器内原有文件	✅ 保留（除非 Volume 有同名文件）	❌ 被清空（完全替换为空目录）
Volume/Bind Mount 为空	容器目录保持不变	容器目录变空
推荐用法	生产环境（持久化+避免意外覆盖）	开发环境（需要完全控制目录）

最佳实践：

- 需要持久化数据（如数据库、日志）：用 Volume。
- 需要完全控制容器目录（如开发时同步代码）：用 Bind Mount。
- 避免用 Volume 挂载空目录来“清空”容器目录（可能达不到预期效果）。

Docker Network

Docker Network（网络）是 Docker 的核心功能之一，它允许容器之间、容器与宿主机、容器与外部网络之间进行通信。Docker Network 通过虚拟网络技术隔离或互联容器，提供灵活的网络配置选项，适用于开发、测试和生产环境。

1. Docker Network 的核心作用

1.1 容器间通信

- 默认隔离：**Docker 容器默认运行在独立的网络命名空间（Network Namespace）中，彼此隔离，无法直接通信。
- 网络互联：**通过 Docker Network，容器可以共享同一虚拟网络，实现互相访问（如通过容器名或 IP）。

1.2 隔离与安全

- 网络隔离**：不同 Docker Network 的容器无法直接通信，增强安全性。
- 安全组**：可结合 `--network` 和 `--ip` 限制容器访问权限（如仅允许特定端口通信）。

1.3 外部网络访问

- 端口映射**：通过 `-p` 暴露容器端口到宿主机（如 `-p 80:8080`）。
- NAT 转换**：容器通过宿主机 IP 访问外部网络（如互联网）。

1.4 DNS 解析

- 内置 DNS**：Docker 提供内置 DNS 服务，容器可通过容器名解析 IP（如 `redis-master` → `172.18.0.2`）。
- 避免硬编码 IP**：减少因容器重启导致 IP 变化带来的配置问题。

1.5 多主机网络（高级场景）

- Overlay 网络**：在 Docker Swarm 或 Kubernetes 集群中实现跨主机容器通信。
- VXLAN 隧道**：通过 VXLAN 封装数据包，实现跨子网通信。

2. Docker Network 的类型

Docker 提供四种网络驱动（Network Driver），默认使用 `bridge`：

网络类型	作用	适用场景	示例命令
bridge	默认网络，容器通过 NAT 访问外部网络	单主机开发、测试	<code>docker network create my-bridge</code>
host	容器直接使用宿主机网络（无隔离）	需要直接访问宿主机资源（如数据库）	<code>--network host</code>
overlay	跨主机网络（Swarm/K8s 集群）	分布式应用、微服务	<code>docker network create --driver overlay my-overlay</code>
none	完全隔离，容器无网络访问	安全隔离测试	<code>--network none</code>

2.1 Bridge 网络（默认）

- 特点
 - 容器通过虚拟网桥（如 `docker0`）互联。
 - 每个容器分配独立 IP（如 `172.17.0.2/16`）。
 - 支持端口映射（`-p`）。
- 示例

```
bash# 创建自定义 bridge 网络
docker network create my-bridge

# 启动容器并加入网络
docker run -d --name web --network my-bridge -p 80:80 nginx
```

2.2 Host 网络

- 特点
 - 容器直接使用宿主机网络栈（无 NAT 转换）。
 - 性能最高，但完全无隔离（容器与宿主机共享端口）。
- 示例

```
bash

docker run -d --name my-app --network host nginx
```

- **访问方式**：直接通过宿主机 IP 和端口访问（如 `http://localhost:80`）。

2.3 Overlay 网络

- 特点
 - 跨主机网络，用于 Docker Swarm 或 Kubernetes。
 - 通过 VXLAN 隧道实现容器间通信。
 - 示例
- (Swarm 集群)：

```
bash# 初始化 swarm 集群
docker swarm init

# 创建 overlay 网络
docker network create --driver overlay my-overlay

# 启动服务并加入网络
docker service create --name web --network my-overlay -p 80:80 nginx
```

2.4 None 网络

- 特点
 - 容器无任何网络接口（仅 `lo` 回环接口）。
 - 需通过 `docker exec` 手动交互。
- 示例

```
bash

docker run -d --name isolated --network none alpine tail -f /dev/null
```

4. 常见使用场景

4.1 微服务架构

- **场景：**多个服务（如 Web、API、数据库）需要互联但隔离。
- **方案**
 - 创建独立网络（如 `frontend-net`、`backend-net`）。
 - 将 Web 和 API 加入 `frontend-net`，数据库加入 `backend-net`。
 - 通过 `frontend-net` 访问 `backend-net`（需路由配置）。

4.2 多主机集群（Swarm/K8s）

- **场景：**跨主机部署容器（如 Redis 集群）。
- **方案**
 - 使用 `overlay` 网络实现容器间通信。
 - 通过 Docker Swarm 或 Kubernetes 管理服务发现。

4.3 安全隔离

- **场景：**限制容器访问权限（如仅允许数据库访问 API）。
- **方案**
 - 创建独立网络（如 `db-net`、`api-net`）。
 - 通过防火墙规则限制跨网络流量。

5. 总结

特性	说明
容器间通信	通过虚拟网络互联（如 <code>bridge</code> 、 <code>overlay</code> ）
隔离性	不同网络容器无法直接通信
外部访问	通过端口映射（ <code>-p</code> ）或 NAT 转换
DNS 解析	支持容器名解析（避免硬编码 IP）
多主机支持	通过 <code>overlay</code> 网络实现跨主机通信

示例命令

```
bash# 1. 创建自定义 bridge 网络
docker network create my-net

# 2. 启动容器并加入网络
docker run -d --name web --network my-net -p 80:80 nginx

# 3. 验证网络
docker network inspect my-net

# 4. 测试容器间通信
docker exec -it web ping another-container
```

Docker Network 是容器化应用的核心组件，合理使用网络类型（如 `bridge`、`overlay`）和配置（如 IPAM、DNS）可以显著提升应用的灵活性、安全性和可维护性。

6. 搭建redis集群

```
docker network create redisnet

#主节点
docker run -d -p 6379:6379 \
-v /btredis/rd1:/bitnami/redis/data \
-e REDIS_REPLICATION_MODE=master \
-e ALLOW_EMPTY_PASSWORD=yes \
--network redisnet --name redis6379 \
bitnami/redis

#从节点
docker run -d -p 6380:6379 \
-v /btredis/rd2:/bitnami/redis/data \
-e REDIS_REPLICATION_MODE=slave \
-e REDIS_MASTER_HOST=redis6379 \
-e REDIS_MASTER_PORT_NUMBER=6379 \
-e ALLOW_EMPTY_PASSWORD=yes \
--network redisnet --name redis6380 \
bitnami/redis
```

Docker Compose

Docker Compose 是 Docker 官方提供的容器编排工具，用于通过 **YAML 文件**（`docker-compose.yml`）定义和运行多容器应用。它简化了复杂应用的部署流程，支持一键启动、停止和管理多个关联容器，适用于开发、测试和轻量级生产环境。

1. Docker Compose 的核心作用

1.1 解决什么问题？

- **多容器依赖**：如 Web 服务 + 数据库 + 缓存（Redis）需要同时运行。
- **配置复杂**：手动执行多个 `docker run` 命令易出错且难以维护。
- **环境一致性**：确保开发、测试、生产环境使用相同的配置。

1.2 核心功能

- **定义服务**：在 YAML 文件中声明容器（如 `web`、`db`）。
- **一键部署**：通过 `docker-compose up` 启动所有容器。
- **依赖管理**：自动处理容器启动顺序（如数据库先启动，Web 服务后启动）。
- **网络隔离**：默认创建独立网络，容器间通过服务名互相访问（如 `db` 代替 IP）。
- **数据持久化**：通过 `volumes` 挂载宿主机目录或卷。

2. Docker Compose 使用步骤

2.1 安装 Docker Compose

- Linux/macOS
：通常随 Docker Desktop 安装，或通过以下命令安装：

```
bashsudo curl -L
"https://github.com/docker/compose/releases/download/v2.23.0/docker-
compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
```

2.2 基本命令

命令	作用
<code>docker-compose up</code>	启动所有服务（前台运行）
<code>docker-compose up -d</code>	后台运行（推荐）
<code>docker-compose down</code>	停止并删除容器、网络
<code>docker-compose ps</code>	查看运行中的容器
<code>docker-compose logs</code>	查看容器日志
<code>docker-compose build</code>	构建镜像（如果定义了 <code>build</code> ）

3. 实战案例：WordPress + MySQL

3.1 场景描述

- 部署一个

WordPress 博客系统,包含:

- **Web 服务:** WordPress 容器 (基于 `wordpress:latest` 镜像)。
- **数据库:** MySQL 容器 (基于 `mysql:8.0` 镜像)。

- 需求

- 数据持久化 (数据库和 WordPress 上传文件)。
- 容器间通过服务名通信 (`wordpress` 访问 `db`)。

3.2 项目结构

```
wordpress-compose/
├── docker-compose.yml # 配置文件
├── data/               # 持久化数据目录 (可选)
│   ├── mysql/         # MySQL 数据存储
│   └── wordpress/     # WordPress 上传文件
```

3.3 编写 `docker-compose.yml`

```
services:
  db:
    image: mysql:8.0          # 使用官方 MySQL 镜像
    container_name: mysql_db  # 自定义容器名
    restart: always           # 自动重启策略
    environment:
      MYSQL_ROOT_PASSWORD: root123 # MySQL root 密码
      MYSQL_DATABASE: wordpress    # 预创建数据库
      MYSQL_USER: wordpress        # 数据库用户
      MYSQL_PASSWORD: wordpress123 # 用户密码
    volumes:
      - ./data/mysql:/var/lib/mysql # 持久化数据库数据
    networks:
      - wp-net                       # 加入自定义网络

  wordpress:
    image: wordpress:latest    # 使用官方 WordPress 镜像
    container_name: wordpress_app
    restart: always
    ports:
      - "8080:80"              # 宿主机 8080 映射到容器 80
    environment:
      WORDPRESS_DB_HOST: db      # 连接 MySQL 服务 (通过服务名 db)
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: wordpress123
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - ./data/wordpress:/var/www/html/wp-content # 持久化上传文件
    depends_on:
      - db
```

```
- db
networks:
  - wp-net

networks:
  wp-net:                # 定义自定义网络
    driver: bridge
```

3.4 启动服务

```
bash# 1. 进入项目目录
cd wordpress-compose

# 2. 启动服务（后台运行）
docker compose up -d

# 3. 验证容器状态
docker-compose ps
```

- 访问 WordPress：浏览器打开 `http://localhost:8080`，按提示完成安装。

3.5 常用操作

操作	命令
停止服务	<code>docker-compose down</code>
查看日志	<code>docker-compose logs -f wordpress</code>
进入容器	<code>docker-compose exec wordpress bash</code>
重建镜像（修改配置后）	<code>docker-compose up -d --build</code>

4. Docker Compose 文件详解

4.1 关键字段

字段	作用	示例
<code>version</code>	Compose 文件版本	<code>'3.8'</code>
<code>services</code>	定义服务（容器）	<code>web: { ... }</code>
<code>image</code>	指定镜像	<code>image: nginx:latest</code>
<code>build</code>	从 Dockerfile 构建镜像	<code>build: ./app</code>
<code>ports</code>	端口映射	<code>"8080:80"</code>
<code>volumes</code>	数据持久化	<code>./data:/var/lib/mysql</code>
<code>environment</code>	环境变量	<code>MYSQL_ROOT_PASSWORD: root123</code>

字段	作用	示例
<code>depends_on</code>	依赖服务	<code>depends_on: - db</code>
<code>networks</code>	加入网络	<code>networks: - wp-net</code>

4.2 网络模式

- **默认**：自动创建 `项目名_default` 网络，容器间通过服务名通信。
- **自定义网络**：通过 `networks` 字段定义（如 `wp-net`）。
- **外部网络**：连接宿主机已有网络（如 `external: true`）。

4.3 数据持久化

- 宿主机目录挂载

```
yamlvolumes:  
- ./data:/var/lib/mysql
```

- 命名卷（推荐生产环境）

```
yamlvolumes:  
mysql_data: # 自动创建命名卷
```

5. 总结

5.1 核心优势

- **简化配置**：通过 YAML 文件统一管理多容器。
- **快速部署**：一键启动复杂应用。
- **环境隔离**：默认网络隔离，避免冲突。
- **数据持久化**：支持目录挂载和命名卷。

5.2 适用场景

- **开发环境**：快速搭建本地服务（如 WordPress、LAMP）。
- **测试环境**：模拟生产环境的多服务依赖。
- **轻量级生产**：单主机部署小型应用（如内部工具）。